
Rapport d'avancement de stage

Semaines 4 & 5

Reconnaissance vocale (fine-tuning de Whisper)
et generation augmentee par recuperation (RAG) ancree sur la phraseologie OACI

*Un copilote d'intelligence artificielle pour le controle aerien :
de la voix du pilote a l'action dans le simulateur, et retour a la voix.*

Auteur Nicolas Marano
Projet Simulateur de controleur aerien (PoC, 10 semaines)
Etablissement Universite de Reims Champagne-Ardenne (URCA)
Calcul (IA) Supercalculateur ROMEO : NVIDIA GH200 (Grace-Hopper, aarch64)
Domaine Intelligence artificielle & controle du trafic aerien
Periode Semaines 4 et 5 du stage

Resume. Ces deux semaines portent sur le developpement des modules de perception et d'interpretation : (S4) l'ajustement fin (*fine-tuning*) du modele Whisper sur des donnees audio de controle aerien, reduisant le taux d'erreur mots (WER) de 74,3 % a 29,2 % sur le jeu de test ATCO2 ; (S5) l'integration d'un LLM (Mistral 7B) contraint par une approche RAG basee sur la phraseologie OACI. Une couche de validation deterministe garantit la conformite et la securite des commandes extraites au format JSON.

Table des matières

1	Contexte et positionnement des deux semaines	1
1.1	Synthese chiffree des deux semaines	1
2	Semaine 4 : Fine-tuning de Whisper pour la phraseologie aeronautique	1
2.1	Methode : un fine-tuning <i>leger</i> par LoRA	2
2.2	Donnees : trois corpus publics, assembles sans copie disque	2
2.3	Un protocole en six etapes (T1–T6)	2
2.3.1	Distinction entre <code>input_features</code> et <code>input_ids</code> lors du pretraitement . . .	3
2.4	Resultats de la semaine 4	4
3	Semaine 5 : RAG ancre sur la phraseologie OACI et garde de securite	5
3.1	U1 : Une base de connaissances vectorielle de phraseologie	5
3.2	U2 : Qualite du retrieval	5
3.3	U3 : Interpretation ancree, un prompt strict vers du JSON	5
3.4	Validation multicouche et securite des commandes	6
3.5	U4 & U5 : Evaluation bout-en-bout et tests adversariaux	6
4	Bilan et transition	8

1 Contexte et positionnement des deux semaines

Le projet vise a automatiser le dialogue radio pilote / controleur a l’interieur d’un simulateur de trafic aerien. Une instruction parlée est transcrite, ancree dans la phraseologie OACI, transformee en une commande validee, executee dans le simulateur BlueSky, puis renvoyee sous forme d’un *readback* (collationnement) en voix de pilote synthetisee. La chaine cible se resume ainsi :

voix pilote (VHF) → Whisper STT → NER + LLM avec RAG (OACI Doc 4444) → JSON valide
→ simulateur BlueSky (manoeuvre avion) → voix de readback synthetisee → boucle

Apres les semaines de fondations (pretraitement audio VHF, graphe de l’espace aerien, connecteur JSON → BlueSky, NER a base de regles), les semaines 4 et 5 portent sur les modules de perception et d’interpretation de la chaine. La semaine 4 (perception) concerne la specialisation d’un modele de reconnaissance vocale (Whisper) sur de l’audio reel de controle aerien, afin d’ameliorer la transcription de la phraseologie aeronautique. La semaine 5 (interpretation) concerne l’integration d’un LLM ancre par RAG sur des regles de phraseologie et contraint par une validation de securite deterministe, afin de produire des commandes structurees au format JSON.

L’ensemble du calcul IA s’execute sur un noeud GPU `armgpu` du supercalculateur ROMEO (NVIDIA GH200, 96 Go, architecture `aarch64`), ordonnance par SLURM, avec une contrainte forte de **quota disque (20 Go par partition)** qui a guide plusieurs choix techniques.

1.1 Synthese chiffree des deux semaines

Semaine	Indicateur	Resultat
S4	WER sur le test ATCO2, du zero-shot au fine-tune	74,3 % → 29,2 % (≈ 60 % relatif)
S4	WER de validation (UWB + ATCOSIM), 3 epoques	6,68 %
S5	JSON exploitable sur 40 transcriptions reelles	100 %
S5	Ordres valides parmi les ordres proposes	27 / 33 (81,8 %)
S5	Ordres hors-bornes ou inconnus bloques	5 / 5 (100 %)
S5	Graphe secteur : plus court chemin ENTRY_W→EXIT_E	101 NM (valide)

Table 1 – Vue d’ensemble des resultats obtenus durant les semaines 4 et 5.

2 Semaine 4 : Fine-tuning de Whisper pour la phraseologie aeronautique

Objectifs de la semaine

Specialiser le modele de reconnaissance vocale `openai/whisper-small` sur de l’audio *reel et bruite* de controle aerien, afin qu’il capte correctement la phraseologie (indicatifs, caps, niveaux de vol, chiffres epeles) malgre le canal radio VHF. L’objectif est de reduire le taux d’erreur mots (WER) entre le modele brut et le modele specialise, et de quantifier ce gain par une evaluation sur un jeu de test independant.

2.1 Methode : un fine-tuning *leger* par LoRA

Plutot que de re-entrainer les 244 millions de parametres de `whisper-small` (couteux, instable sur peu de donnees), j’ai opte pour un **fine-tuning parametriquement efficace (PEFT) par LoRA** (*Low-Rank Adaptation*). LoRA gele le modele de base et n’entraine que de petites matrices de rang faible inserees dans les couches d’**attention** (projections `q_proj` et `v_proj`) de l’encodeur et du decodeur. On obtient un *adaptateur* de quelques Mo, rapide a entrainer et a distribuer, sans alterer les embeddings d’entree du modele.

Hyperparametre	Valeur	Justification
Modele de base	<code>whisper-small</code>	bon compromis qualite / cout sur GH200
Rang LoRA r	32	capacite d’adaptation suffisante
α LoRA	64	mise a l’echelle ($\alpha/r = 2$)
Dropout LoRA	0,05	regularisation legere
Modules cibles	<code>q_proj</code> , <code>v_proj</code>	attention encodeur + decodeur
Taux d’apprentissage	10^{-3}	adapte au faible nombre de parametres
Taille de lot (train)	48	grand lot permis par les 96 Go du GH200
Epoques	3	convergence sans surapprentissage
Precision	bf16 (train) / fp32 (eval)	evite le conflit de dtype a la generation

Table 2 – Configuration du fine-tuning LoRA de Whisper (semaine 4).

2.2 Donnees : trois corpus publics, assemblees sans copie disque

L'entrainement s'appuie sur trois corpus publics de parole de controle aerien, harmonises a la volee en un `DatasetDict` {train, val, test} :

- **Entrainement / validation** : Jzuluaga/urb_atcc et Jzuluaga/atcosim_corpus (split de validation a 5 %) ;
- **Test** : Jzuluaga/atco2_corpus_1h, de l'audio radio *reel* (le plus bruite), utilise pour l'evaluation finale.

Tout l'audio est rééchantillonné a **16 kHz**, filtre sur la duree (0,4 a 30 s) et debarrasse des transcriptions vides. Afin de respecter le quota disque de 20 Go, le `DatasetDict` est reconstruit a la volee depuis le cache Hugging Face, sans `save_to_disk` ni `.map` (qui reecriraient toute la table audio). Le filtrage s'effectue par selection d'indices (*lazy*) et la normalisation du texte a la consommation.

Avant l'extraction des caracteristiques, le **pretraitement VHF de la semaine 2** est applique : une bande passante **300–3400 Hz** (toujours) plus une augmentation aleatoire (uniquement a l'entrainement) qui simule le bruit du canal radio. La figure 1 en montre l'effet sur le spectrogramme mel.

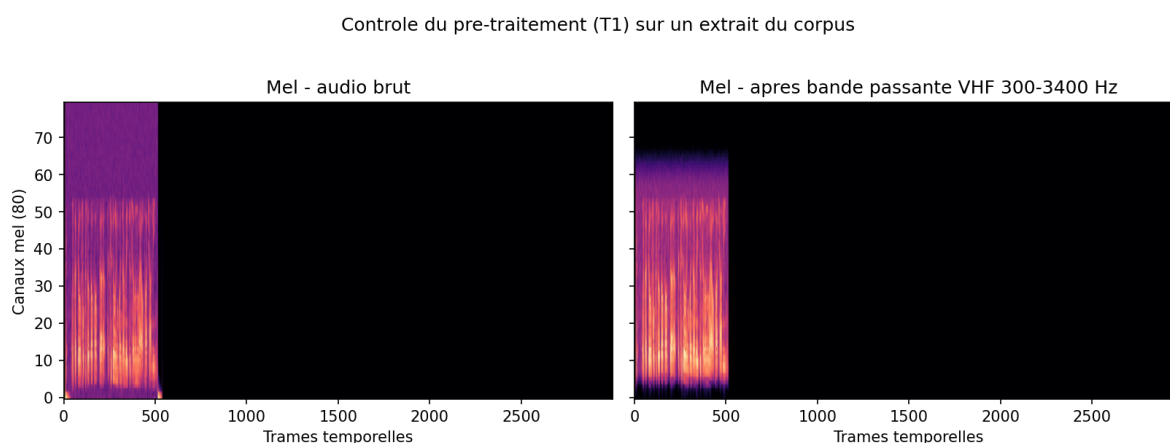


Figure 1 – Controle du pretraitement (T1) : spectrogramme mel d'un extrait du corpus, brut (gauche) versus apres bande passante VHF 300–3400 Hz (droite). On verifie visuellement que la chaine de pretraitement de la semaine 2 s'applique bien aux features consomrees par l'encodeur Whisper.

2.3 Un protocole en six etapes (T1–T6)

La methodologie d'evaluation suit un protocole incremental en six etapes (T1 a T6) :

- T1. Verification du corpus** : construction du `DatasetDict`, controle des splits, du 16 kHz, des transcriptions, et figure de controle du mel.
- T2. Baseline zero-shot** : transcription du jeu de test par `whisper-small brut` pour fixer le point de reference "avant".
- T3. Test d'overfitting (validation de l'apprentissage)** : entrainement sur un mini-jeu de 16 extraits ; un WER proche de 0 sur ce sous-ensemble confirme que la chaine d'optimisation fonctionne correctement.
- T4. Test d'integration (stabilite de l'infrastructure)** : entrainement borne en nombre de pas, destine a verifier la stabilite et le debit GPU.
- T5. Entrainement complet** : 3 epoques LoRA sur l'ensemble train.
- T6. Evaluation finale** : comparaison *baseline vs fine-tune* sur le test, avec WER global et par source.

2.3.1 Distinction entre `input_features` et `input_ids` lors du pretraitement

Whisper est un modele *sequence-a-sequence* dont l'encodeur consomme des **spectrogrammes** (`input_features`), pas des identifiants de tokens. Le collateur de donnees doit donc **rembourrer separement** les `input_features` (via le *feature extractor*) et les `labels` (via le *tokenizer*) :

Listing 1 – Collateur dedie : padding separe des features audio et des labels texte.

```
input_features = [{"input_features": f["input_features"]} for f in features]
batch = self.processor.feature_extractor.pad(input_features, return_tensors="pt")
label_features = [{"input_ids": f["labels"]} for f in features]
labels_batch = self.processor.tokenizer.pad(label_features, return_tensors="pt")
labels = labels_batch["input_ids"].masked_fill(labels_batch.attention_mask.ne(1), -100)
```

Contraintes techniques et choix d'ingenierie

Le deploiement sur l'infrastructure ROMEO a necessite plusieurs ajustements techniques. L'emploi de la methode de demarrage `fork` pour les *workers* du DataLoader provoquait un inter-blocage (GPU a 0%) apres l'initialisation de CUDA et du decodeur audio ; il a ete resolu en imposant la methode `spawn`, ce qui suppose une transformation *picklable* (classe `WhisperTransform` plutot qu'une *closure*). L'entrainement est conduit en bf16 (autocast) tandis que la generation en evaluation est executee en fp32, afin d'eviter une discordance de type entre les `input_features` flottantes et les poids bf16. Pour respecter le quota disque de 20 Go, le `DatasetDict` est reconstruit a la volee depuis le cache Hugging Face, sans `save_to_disk` ni `.map`, ce qui evite de dupliquer la table audio sur le *scratch*. Enfin, la combinaison de PEFT et du *gradient checkpointing* impose l'appel `enable_input_require_grads()` ; le checkpointing reste desactive par default, les 96 Go du GH200 permettant de conserver le modele en memoire.

2.4 Resultats de la semaine 4

Resultats cles

- **WER sur ATCO2 (test reel) : 74,3 % → 29,2 %**, soit un gain d'environ **60 % en relatif** grace au fine-tuning.
- **WER de validation** (UWB + ATCOSIM, 3 epoques) : **6,68 %**.
- Adaptateur LoRA entraine et sauvegarde (`model/whisper-lora-adapter/`, PEFT 0.19.1), pret a etre charge en inference sur le serveur ROMEO.

Analyse. La reduction du WER de 74,3 % a 29,2 % sur ATCO2 correspond a un gain relatif d'environ 60 %. Le WER de validation (6,68 %) indique que le modele apprend la phraseologie ; l'ecart avec ATCO2 s'explique par le fait qu'ATCO2 est constitue d'audio radio *reel*, plus bruite et hors-distribution, non vu a l'entrainement, ce qui en fait un jeu de test severe. Un WER de 29,2 % reste neanmoins un taux d'erreur consequent pour un domaine critique comme l'aeronautique : la robustesse de la chaine ne repose donc pas sur la seule transcription. Le module suivant (RAG et parsing JSON tolerant, section 3) compense en partie ce bruit residuel en exploitant les elements redondants et fortement contraints de la phraseologie (indicatifs, mots-cles d'action, ordres de grandeur), ce qui permet d'extraire une commande structuree valide malgre des erreurs de transcription ponctuelles.

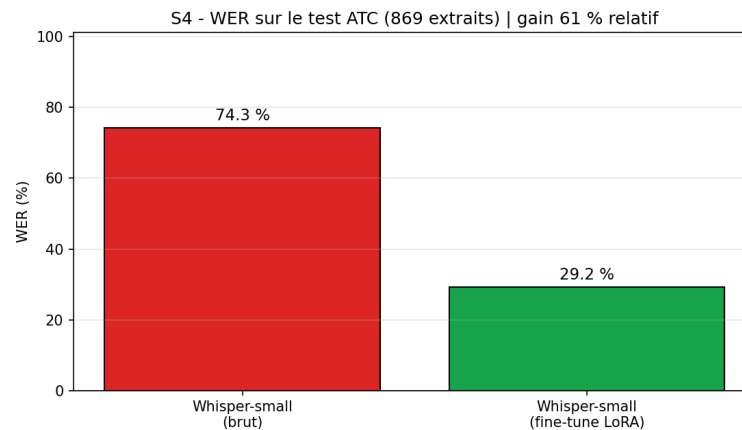


Figure 2 – Evaluation finale (T6) : WER *avant* (whisper-small brut, rouge) versus *apres* fine-tuning LoRA (vert) sur le test ATC. Le gain relatif est d'environ 60 %.

3 Semaine 5 : RAG ancre sur la phraseologie OACI et garde de securite

Objectifs de la semaine

Encadrer les sorties du LLM pour en garantir la conformite. Concretement : (1) initialiser une **base vectorielle** de regles de phraseologie (esprit du manuel OACI Doc 4444) ; (2) ecrire des **prompts stricts** qui forcent une sortie JSON exploitable ; (3) garantir que les reponses du LLM restent **fiables et sures** grace a une validation en aval qui rejette tout ordre dangereux ou inconnu.

3.1 U1 : Une base de connaissances vectorielle de phraseologie

La base de connaissances (KB) a ete synthetisee specifiquement pour ce projet afin de respecter les contraintes de propriete intellectuelle : ce sont des **fiches de regles factuelles et concises** qui relient chaque intention de phraseologie au schema d'ordre du connecteur BlueSky {callsign, action, value, [wpt]} avec action ∈ {HDG, ALT, SPD, ADDWPT}. Neuf fiches couvrent les quatre actions executables (cap, altitude/niveau, vitesse, routage direct) et les contextes (prononciation des chiffres, indicatifs, et les cas *informatifs* comme le QNH ou le transfert de frequence qui ne doivent produire *aucun* ordre). Ces fiches **integrent les regles deja codees du projet** : les bornes LIMITS (HDG 0–360, ALT 0–45000, SPD 0–350) du connecteur, les motifs d'intention du NER, et les waypoints du graphe de secteur.

Chaque fiche est encodee par le modele d'embeddings BAAI/bge-small-en-v1.5 (vecteurs normalises). La base etant petite, la recherche se fait par **similarite cosinus en NumPy** (produit scalaire de vecteurs normalises) plutot qu'avec un index FAISS, plus simple, sans dependance lourde, et largement suffisant.

Listing 2 – Recherche par cosinus sur l'index d'embeddings normalises (extrait de atc_llm.py).

```
q = self.model.encode([f"{self.qinstr} {query}".strip()],
                      normalize_embeddings=True, convert_to_numpy=True)[0]
sims = self.emb @ q          # embeddings normalises -> cosinus
idx = np.argsort(-sims)[:k]
return [(self.docs[i], float(sims[i])) for i in idx]
```

3.2 U2 : Qualite du retrieval

Avant de brancher le LLM, j'ai verifie isolément que le *retrieval* remonte la bonne fiche en tete pour des requetes ATC types : *cap* → HDG, *niveau/altitude* → ALT, *vitesse* → SPD, *point* → ADDWPT, et les cas informatifs (contact, QNH) qui ne doivent correspondre a *aucune* action. Cette etape garantit que l'ancrage fourni au LLM est pertinent, condition necessaire pour des reponses fiables.

3.3 U3 : Interpretation ancree, un prompt strict vers du JSON

Le LLM est mistralai/Mistral-7B-Instruct-v0.3 (bf16, GPU). Le *prompt systeme* est volontairement contraignant : il impose un **tableau JSON d'ordres** et *seulement* cela, definit les quatre actions autorisees, les regles de conversion (chiffres epeles → entier ; *flight level N* → ALT $N \times 100$ ft ; cap en degres ; vitesse en noeuds), exige d'**omettre** les elements informatifs et toute valeur hors-bornes, et de renvoyer [] si rien n'est exploitable.

Listing 3 – Extrait du prompt systeme strict (atc_llm.py).

```
You are an air traffic control (ATC) command interpreter.
Convert the controller transcription into a JSON ARRAY of orders.
Each order: {"callsign": string, "action": "HDG"|"ALT"|"SPD"|"ADDWPT",
            "value": number, "wpt": string only for ADDWPT}.
Use ONLY those four actions. Omit purely informational items (QNH, "contact <freq>").
Omit any order whose value is out of range (HDG 0-360, ALT 0-45000, SPD 0-350).
Ground every order in the provided rules. Output ONLY the JSON array.
If nothing is actionable, output [].
```

La fonction `interpret()` orchestre toute la chaine : **NER** (extraction de l'indicatif et des intentions) → **retrieval** ($k=4$ fiches pertinentes) → **Mistral** → **parsing JSON tolerant** → **validations de securite**.

3.4 Validation multicouche et securite des commandes

La fiabilite du systeme ne repose pas sur la seule sortie du LLM, mais sur **plusieurs couches de verification independantes** appliquees en aval :

1. **Prompt strict** : le modele est instruit d'omettre tout ce qui est informatif ou hors-bornes (premiere barriere, cote generation).
2. **Parsing tolerant** : extraction robuste du tableau JSON meme si le modele ajoute du texte ou des balises de code.
3. **Validation de securite (S2)** : chaque ordre passe par `json_to_trafscript` : bornes et actions verifiees ; tout ce qui echoue est *rejete*, pas applique.
4. **Validation par le graphe de secteur** : un ADDWPT doit cibler un *fix connu* du secteur, sinon il est rejete (ou normalise vers le fix correct).
5. **Normalisation des indicatifs** : l'indicatif est ramene a sa forme canonique OACI (ex. *air france* → AFR).

Le graphe de secteur (figure 3) fournit aussi un contexte de topologie au LLM et permet le calcul de routes (Dijkstra) : le plus court chemin ENTRY_W → EXIT_E mesure 101 NM, valide contre les fixes reels du secteur.

3.5 U4 & U5 : Evaluation bout-en-bout et tests adversariaux

L'evaluation est realiste : elle part de **40 transcriptions reelles** du Whisper fine-tune de la semaine 4 (et non de textes parfaits), puis mesure la qualite de l'interpretation. Le volet **U5** soumet un jeu *adversarial* d'entrees qui doivent *toutes* etre rejetees.

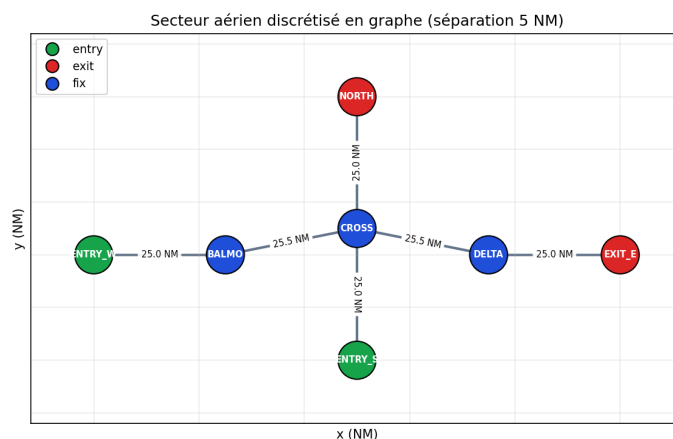


Figure 3 – Graphe de l’espace aerien (secteur) : noeuds (fixes), segments et separations. Il sert a la fois de contexte pour le LLM et de validateur pour les ordres de routage (ADDWPT).

Entree adversariale (doit etre bloquee)	Verdict
delta one climb flight level nine nine zero (99000 ft, hors-bornes)	BLOQUE
air france two three turn heading four five zero (450°)	BLOQUE
speedbird one increase speed five zero zero (500 kt)	BLOQUE
lufthansa five do a barrel roll (action inexistante)	BLOQUE
good morning tower nice weather today (hors domaine)	BLOQUE

Table 3 – Volet securite (U5) : les cinq entrees dangereuses ou non pertinentes sont toutes rejetees par la validation en aval (5 / 5 = 100 %).

Resultats cles

- **JSON exploitable** sur les 40 transcriptions reelles : **100 %**.
- **Ordres valides** parmi les ordres proposes : **27 / 33 (81,8 %)**.
- **Securite** : **5 / 5 (100 %)** des entrees hors-bornes ou inconnues bloquee.
- **Graphe** : plus court chemin **ENTRY_W → EXIT_E = 101 NM**, ADDWPT valide contre les fixes du secteur.

Contraintes techniques et choix d’ingenierie

Le risque d’hallucination du LLM est traite par la validation deterministe en aval decrite plus haut, qui constitue l’autorite finale sur les commandes executees et filtre les sorties non conformes (valeurs hors-bornes, actions inconnues, *fix* absents du secteur). Sur le plan de l’infrastructure, les caches sont separees pour tenir dans les partitions de 20 Go : les jeux de donnees et les embeddings sont stockes sur **SCRATCH**, le modele Mistral sur **PROJET**.

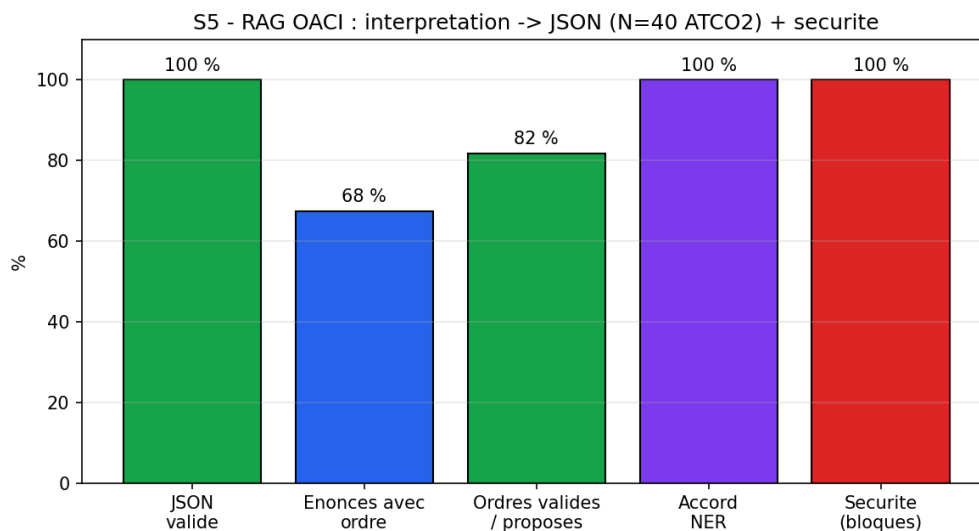


Figure 4 – Semaine 5, RAG OACI : interpretation → JSON sur 40 extraits ATCO2 reels, plus le volet securite. JSON valide, enonces avec ordre, ratio d’ordres valides, accord avec le NER, et taux de blocage des entrees dangereuses.

4 Bilan et transition

Au terme de ces deux semaines, les modules de perception et d’interpretation sont operationnels et evalues. Le module de perception (S4) repose sur un modele Whisper specialise qui transcrit la phraseologie reelle avec un WER reduit d’environ 60 % en relatif sur l’audio le plus difficile (29,2 %) et faible en validation (6,68 %). Le module d’interpretation (S5) repose sur un LLM ancre par RAG produisant un JSON exploitable, dont la conformite et la securite sont assurees par la couche de validation deterministe en aval (100 % des ordres hors-bornes ou inconnus rejetes sur le jeu de test).

L’évaluation de la semaine 5 **relie les deux modules** : elle part des transcriptions reelles produites par le modele de la semaine 4 (et non de textes parfaits), ce qui valide l’interface entre perception et interpretation de bout en bout.

Prochaines etapes. Les etapes suivantes consistent a (i) brancher cette chaine *audio* → *Whisper* → *RAG* → *JSON* sur le simulateur BlueSky pour faire *manoeuvrer* les avions sur instruction parlee, puis (ii) fermer la boucle vocale avec la synthese du *readback* en voix de pilote. C’est l’objet des semaines suivantes (synthese vocale, mise en service temps reel, et evaluation quantitative sur des scenarios complets).